



PPSC - Rendu Vincent Lannelongue

Status	Done
Dates	July 7, 2026

All plots can be generated with 'plot_results.py' and are stored in the './results' dir.

- **Question 1:**

Cf code.

- **Question 2:**

From the serial implementation the results should not change when parallelising. The unit tests verify this, while being light and fast to run. This allows to easily test in the parallelisation added unforeseen bugs.

- **Question 3:**

The Machin approximation shows much faster convergence when n is increasing (see plot `method_0.svg`).

- **Question 4:**

Process 0 generates the whole vectors with all terms, it would be much more efficient to let all processes compute their own terms, compute partial sums and regroup everything in the end. This implementation might create a memory problem (storing all values by process 0), and maybe a bandwidth bottleneck (process 0 needs to share all values to other processes).

- **Question 5:**

The MPI_Reduce call combined with the MPI_Sum operation allowed to regroup and sum all partial sums to a single global sum variable that we could exploit from

the root process. The `MPI_Init`, `MPI_Finalize`, `MPI_Comm_rank`, `MPI_Comm_size` and `MPI_Abort` seemed necessary for our main purpose.

Regarding the execution time, the `MPI_WTime` function was used, to time from the initialisation to the end of the computation. The `MPI_Init` is not measured this way, but the rest of the processes are. The plots show that for $n = 1$ (and $n = 1000$ for the Zeta function), augmenting the number of processes actually increases the computation time.

When fixing n , the Machin formula is still clearly more accurate. Even though at a given n , Zeta is much faster to compute, when comparing both methods with a fixed time budget, Machin is still the better option.

RQ: for the Machin formula, computing with the `partial_pi` function (single sum) is slightly slower (3-10%) than using 2 separate sums to compute both term of the formula, and regroup them in the root process. I kept the fastest method.

- **Question 6:**

As shown in the plots (`method_1_error.svg` and `method_1_time.svg`) we observe for the Zeta method a discrepancy at $n = 10^9$ for the errors. Perhaps rounding errors with the number of sums varying with the number of procs, when the error get's low enough.

- **Question 7:**

As expected the results are exactly the same between both methods. Time-wise, the doubling method is consistently faster (plot `reduc_time.svg`).

- **Question 8:**

Error and time plots similar to question 5 have been generated, with 2 threads for all experiments. Errors don't change, but with OpenMP the computation is slower, especially with a high number of processors.

- **Question 9:**

Everything still functions as expected with the combination. MPI alone won't take advantage of the shared node memory and add some overhead when sending data between processes and cores. OpenMP fixes that, by allowing shared memory usage.

- **Question 10:**

To compute the series: for Zeta (resp. Machin), each term requires 2 operations (resp. 5) times n for each term, and n to sum the terms (resp. $2n$, times 2 for the 2 sums). Total: $3n$ (resp. $12n$).

- **Question 11:**

Depending on the accuracy we need, as we saw that parallel processing became more efficient than serial programming after values of n sufficiently large.

The problem however is well suited for parallel programming: the series can easily be partitioned and partially computed before being regrouped.